Tutorial -11(03-11-2026)

1.A college wants to develop a student information system in which students can request their examination results from a central server. Develop a Python-based TCP client–server application in which the client sends a student's register number to the server, and the server retrieves and sends the corresponding result to the client.

```
import socket
import sys
import threading

HOST = "127.0.0.1"
PORT = 5000

# Sample database: register number -> (name, subject marks, result)
RESULTS = {
    "RA2511003010001": ("Arun Kumar",  {"Maths": 88, "Physics": 76, "Python": 92}, "PASS"),
    "RA2511003010002": ("Priya Sharma", {"Maths": 95, "Physics": 89, "Python": 97}, "PASS"),
    "RA2511003010003": ("Ravi Teja",    {"Maths": 34, "Physics": 41, "Python": 55}, "FAIL"),
    "RA2511003010004": ("Meena Devi",   {"Maths": 72, "Physics": 68, "Python": 81}, "PASS"),
}


def build_result(reg_no):
    record = RESULTS.get(reg_no)
    if record is None:
        return f"No result found for register number: {reg_no}"
    name, marks, status = record
    lines = [f"Register No : {reg_no}", f"Name        : {name}"]
    for subject, mark in marks.items():
        lines.append(f"{subject:<12}: {mark}")
    lines.append(f"Total       : {sum(marks.values())}/{len(marks) * 100}")
    lines.append(f"Result      : {status}")
    return "\n".join(lines)


def handle_client(conn, addr):
    print(f"[+] Connected: {addr}")
    try:
        while True:
            data = conn.recv(1024)
            if not data:
                break
```

```
            reg_no = data.decode().strip().upper()
            if reg_no == "EXIT":
                break
            print(f"[{addr[1]}] Request for: {reg_no}")
            conn.sendall(build_result(reg_no).encode())
    except ConnectionError:
        pass
    finally:
        conn.close()
        print(f"[-] Disconnected: {addr}")


def run_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(5)
    print(f"Server listening on {HOST}:{PORT} (Ctrl+C to stop)")
    try:
        while True:
            conn, addr = server.accept()
            threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()
    except KeyboardInterrupt:
        print("\nServer stopped.")
    finally:
        server.close()


def run_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        client.connect((HOST, PORT))
    except ConnectionRefusedError:
        print("Cannot connect. Start the server first.")
        return
    print("Connected to result server. Type 'exit' to quit.")
    try:
        while True:
            reg_no = input("\nEnter Register Number: ").strip()
            if not reg_no:
                continue
            if reg_no.lower() == "exit":
                client.sendall(b"EXIT")
                break
```

```
            client.sendall(reg_no.encode())
            print("\n" + client.recv(4096).decode())
    except (KeyboardInterrupt, ConnectionError):
        pass
    finally:
        client.close()
        print("Connection closed.")


if __name__ == "__main__":
    mode = sys.argv[1].lower() if len(sys.argv) > 1 else input("Run as (server/client):
").strip().lower()
    if mode == "server":
        run_server()
    elif mode == "client":
        run_client()
    else:
        print("Usage: python result_system.py [server|client]")
```

2.A weather monitoring station collects temperature readings from multiple sensors and transmits them to a central monitoring server. Develop a Python-based UDP client–server application in which the client sends temperature readings to the server, and the server receives and displays the readings. Justify the suitability of UDP for this application compared with TCP.

```
import socket
import random
import time
import sys

HOST = "127.0.0.1"
PORT = 6000


def run_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server.bind((HOST, PORT))
    print(f"Weather server listening on {HOST}:{PORT} (Ctrl+C to stop)")
    try:
        while True:
            data, addr = server.recvfrom(1024)
            message = data.decode().strip()
            try:
```

```
            sensor_id, temperature = message.split(",")
            print(f"Sensor {sensor_id} from {addr[0]}:{addr[1]} -> {float(temperature):.1f} °C")
        except ValueError:
            print(f"Invalid data from {addr}: {message}")
    except KeyboardInterrupt:
        print("\nServer stopped.")
    finally:
        server.close()


def run_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sensor_id = input("Enter sensor ID (e.g. S1): ").strip() or "S1"
    print(f"Sensor {sensor_id} sending readings (Ctrl+C to stop)")
    try:
        while True:
            temperature = round(random.uniform(20.0, 40.0), 1)
            message = f"{sensor_id},{temperature}"
            client.sendto(message.encode(), (HOST, PORT))
            print(f"Sent: {temperature} °C")
            time.sleep(2)
    except KeyboardInterrupt:
        print("\nSensor stopped.")
    finally:
        client.close()


if __name__ == "__main__":
    mode = sys.argv[1].lower() if len(sys.argv) > 1 else input("Run as (server/client):
").strip().lower()
    if mode == "server":
        run_server()
    elif mode == "client":
        run_client()
    else:
        print("Usage: python weather_udp.py [server|client]")
```

3.A manufacturing company needs to simplify mathematical expressions used in calculating production costs. Develop a Python program using the SymPy library to define symbolic variables, expand and factorize algebraic expressions, simplify the resulting expressions, and solve a quadratic equation to determine the unknown production quantity.

```
from sympy import symbols, expand, factor, simplify, Eq, solve, pprint

q = symbols("q")
a, b = symbols("a b")

print("=" * 55)
print("PRODUCTION COST SIMPLIFICATION USING SYMPY")
print("=" * 55)

cost = (q + 20) * (q - 5) - 10 * (q + 5)
print("\n1. Original cost expression:")
pprint(cost)

expanded = expand(cost)
print("\n2. Expanded expression:")
pprint(expanded)

factored = factor(expanded)
print("\n3. Factorized expression:")
pprint(factored)

ratio = (q**2 + 5 * q - 150) / (q - 10)
print("\n4. Cost per unit expression before simplification:")
pprint(ratio)
print("    After simplification:")
pprint(simplify(ratio))

material = a**2 - b**2
print("\n5. Material cost difference a^2 - b^2 factorized:")
pprint(factor(material))

equation = Eq(expanded, 0)
print("\n6. Break-even quadratic equation:")
pprint(equation)
```

```
solutions = solve(equation, q)
print("\n7. All solutions for q:", solutions)

valid = [s for s in solutions if s > 0]
print("\n8. Valid production quantity (positive root):", valid[0], "units")
```

4.An automobile company wants to analyze the motion of a vehicle using its displacement function s(t) = t^3 - 6t^2 + 9t + 5, where t represents time. represents time. Develop a Python program using the SymPy library to determine the velocity and acceleration through symbolic differentiation, identify the time at which the vehicle is momentarily at rest, and calculate the displacement over a specified time interval using definite integration.

```
from sympy import symbols, diff, solve, integrate, factor, pprint

t = symbols("t", real=True)
s = t**3 - 6*t**2 + 9*t + 5

print("=" * 55)
print("VEHICLE MOTION ANALYSIS USING SYMPY")
print("=" * 55)

print("\n1. Displacement function s(t):")
pprint(s)

v = diff(s, t)
print("\n2. Velocity v(t) = ds/dt:")
pprint(v)
print("   Factorized:")
pprint(factor(v))

a = diff(v, t)
print("\n3. Acceleration a(t) = dv/dt:")
pprint(a)

rest_times = solve(v, t)
```

```
print("\n4. Vehicle is momentarily at rest when v(t) = 0, at t =", rest_times)
for r in rest_times:
    print(f"   t = {r}: displacement = {s.subs(t, r)}, acceleration = {a.subs(t, r)}")

t1, t2 = 0, 4
displacement = integrate(v, (t, t1, t2))
print(f"\n5. Displacement from t = {t1} to t = {t2} (integral of v dt):",
displacement)
print("   Check s(t2) - s(t1) =", s.subs(t, t2) - s.subs(t, t1))

points = [t1] + [r for r in rest_times if t1 < r < t2] + [t2]
distance = sum(abs(integrate(v, (t, points[i], points[i + 1]))) for i in
range(len(points) - 1))
print(f"\n6. Total distance travelled from t = {t1} to t = {t2}:", distance)
```

5.A digital security system validates binary access codes before granting access to users. The system accepts only binary strings that end with "01". Construct a DFA to recognize valid access codes and develop a Python program using the Automata library to implement the DFA. Test the automaton with different binary strings and display whether each string is accepted or rejected.

```
from automata.fa.dfa import DFA

dfa = DFA(
    states={"q0", "q1", "q2"},
    input_symbols={"0", "1"},
    transitions={
        "q0": {"0": "q1", "1": "q0"},
        "q1": {"0": "q1", "1": "q2"},
        "q2": {"0": "q1", "1": "q0"},
    },
    initial_state="q0",
    final_states={"q2"},
)

print("=" * 45)
```

```
print("BINARY ACCESS CODE VALIDATOR (ends with 01)")
print("=" * 45)

test_codes = ["01", "101", "1101", "0001", "10", "11", "0", "1", "1001", "0110",
"10101", "000"]

print(f"\n{'Access Code':<15}{'Result'}")
print("-" * 35)
for code in test_codes:
    result = "ACCEPTED (Access Granted)" if dfa.accepts_input(code) else
"REJECTED (Access Denied)"
    print(f"{code:<15}{result}")

print("\nCheck your own codes (type 'exit' to quit):")
while True:
    code = input("Enter binary code: ").strip()
    if code.lower() == "exit":
        break
    if not code or any(ch not in "01" for ch in code):
        print("Invalid input: use only 0 and 1.")
        continue
    print("ACCEPTED (Access Granted)" if dfa.accepts_input(code) else
"REJECTED (Access Denied)")
```

6.A text processing system is designed to identify strings over the alphabet that contain the substring "ab". Construct an NFA to recognize such strings and develop a Python program using the Automata library to implement the NFA. Test the automaton with different input strings and display whether each string is accepted or rejected.

```
from automata.fa.nfa import NFA

nfa = NFA(
    states={"q0", "q1", "q2"},
    input_symbols={"a", "b"},
    transitions={
```

```
        "q0": {"a": {"q0", "q1"}, "b": {"q0"}},
        "q1": {"b": {"q2"}},
        "q2": {"a": {"q2"}, "b": {"q2"}},
    },
    initial_state="q0",
    final_states={"q2"},
)

print("=" * 50)
print("NFA: STRINGS CONTAINING SUBSTRING 'ab'")
print("=" * 50)

test_strings = ["ab", "aab", "abb", "bab", "baba", "ba", "bb", "aa", "a", "b",
"bbbab", "aaaa"]

print(f"\n{'Input String':<15}{'Result'}")
print("-" * 30)
for s in test_strings:
    print(f"{s:<15}{'ACCEPTED' if nfa.accepts_input(s) else 'REJECTED'}")

print("\nCheck your own strings (type 'exit' to quit):")
while True:
    s = input("Enter string over {a, b}: ").strip()
    if s.lower() == "exit":
        break
    if any(ch not in "ab" for ch in s):
        print("Invalid input: use only a and b.")
        continue
    print("ACCEPTED" if nfa.accepts_input(s) else "REJECTED")
```